



PA 4 Report

2024010779 刘家豪

实现方法

本题实现 DSA decode 阶段的 BF16 indexer score. 官方测试固定为 `Hidx=64`, `Didx=128`, `PageSize=64`, 因此主路径针对这一形状优化, 同时保留通用兜底逻辑.

实现中主要有以下 GPU kernel:

- `_dsa_empty_feature_kernel`: 处理 `Hidx <= 0` 或 `Didx <= 0` 的特殊情况, 对 valid token 写 0, 对 invalid token 写 `-inf`.
- `_dsa_indexer_scores_official_2page_kernel`: official shape 的中等规模路径, 一个 Triton program 处理同一 batch 的 2 个 page.
- `_dsa_indexer_scores_official_group_page_kernel`: 主要优化路径, 一个 Triton program 处理同一 batch 的 4 个 logical pages.
- `_dsa_indexer_scores_kernel`: 通用兜底逻辑, 用 mask 支持非 official shape.

主 kernel 的 grid 为

```
(triton.cdiv(MaxPages, GROUP_PAGES), B)
```

其中 `GROUP_PAGES=4`, 即每个 program 对应一个 batch item 和连续 4 个 logical pages. 在 Triton 中一个 program 对应一个 CTA. case6 (`total_pages == 4096`) 使用 `num_warps=8`, 即 256 个线程/program; 更大的 case 使用 `num_warps=4`, 即 128 个线程/program.

每个 program 首先读取当前 batch 的 `Q[64,128]` 和 `w[64]`, 然后对 group 内每个 logical page:

1. 通过 `block_table[b, logical_page]` 读取 physical page id;
2. 加载对应的 `K[64,128]`;
3. 计算 token-major 的 `K @ Q.T`, 得到 `dots[64,64]`;
4. 对 head 维执行 `ReLU + weight + sum`, 写回 64 个 score;
5. 对 `s >= context_lens[b]` 的位置显式写入 `-inf`.

存储层次方面, Q 和 K tile 经 shared memory staging 后参与矩阵乘, 大约各占 16KB shared memory; w[64] 转成 FP32 后保存在寄存器中; dots[64,64] 使用 FP32 accumulator. context_lens 和 block_table 只产生少量 global load.

该 kernel 使用了 Hopper 上的 WGMMMA/HGMMMA 指令. Triton 的 tl.dot 会 lowering 到 BF16 输入、FP32 accumulate 的 wmma.mma_async / HGMMMA 指令. 离线编译检查显示主 kernel 无 local memory spill, shared memory 用量约为 32KB.

测试点 5, 7, 10 结果

testcase	OJ Baseline	我的实现	加速比
5	4551 us	11 us	413.7x
7	18 ms	46 us	391.3x
10	67 ms	174 us	385.1x

AI 使用说明

本次作业中使用了 Pi Coding Agent + GPT 5.5 优化. 主要用途包括:

- 整理题目约束;
- 辅助生成和筛选 Triton kernel 变体;
- 辅助编写离线编译脚本, 使用 ptxas / nvdiasm 查看寄存器数、shared memory、spill 和 WGMMMA 指令;
- 启动独立 subagent 基于题面重新思考实现方案, 用于交叉验证当前优化方向.

最终保留的优化均通过 OJ 正确性和性能结果筛选; 不符合题面要求的 unsafe 假设均已回退.